

PD Module 1 — Lab Manual: Synthesis (Logical)

Learning Objective

In this lab, synthesizable Verilog code (RTL) is converted to a optimized technology dependent gate level netlist.

Learning Outcome

After completing the lab, you will be able to,

- 1 Setup the project path and specify the file directories.
- 2 Understand link, target libraries and design files.
- 3 Load RTL design, SDC and read the physical constraint (DEF) file.
- 4 Synthesize into optimized and technology dependent gate-level netlist.
- 5 Check for the reports if the design has met timing, area, power, and constraints after synthesis.
- 6 Write out the Netlist and Synopsys design constraints (SDC) files.

Task

Copy the lab database to your home directory.

```
linux> cp -rf /home/pdsa/chiptop2.0/synth ./
```

Go through the directory structure & files.

RTL -> Directory for RTL files

inputs -> Directory for inputs file

logs -> Directory for log files

reports -> Directory to save all reports

scripts -> Directory for script files

outputs -> Directory to save output files

work -> Directory to run the tool

In the scripts directory open, "synth_logical.tcl" using "vi" or "gvim" editor command from the scripts directory and go through the file contents.

```
linux > gvim synth_logical.tcl
```

From a different linux shell, Invoke the Design Compiler (DC) tool using "dc_shell" from the work directory by following steps,

```
linux > cd chiptop/synth_logical/  
linux:synth > cd work  
linux:work > tcsh  
linux:work > dc_shell -output_log_file ../logs/synth_logical.log
```

Set the current project path, link library path, target library path for the design.

Now specify directories that the tool searches for files, by using the below command.

```
dc_shell>source ../scripts/common_setup.tcl
```

Above command sets the link library to be used by the DC to resolve references which are instantiated in the RTL. These could be either PLL, Memory, standard cell, or any other macro for which there is no module definition present in the RTL. For example,

```
BUF uBUF (.FIN(clk), .FOUT(clkout));
```

The "BUF" module shall be resolved from the link library. That is, the DC compiler investigates the link library in order to get the characteristic information of the BUF like timings etc.

Specify the list libraries used in the design, by using the "link_library" command.

```
dc_shell>set link_library $LINK_LIBRARY_FILES_CLG
```

Specify the list of technology libraries, by using the "target_library" command.

```
dc_shell>set target_library $TARGET_LIBRARY_FILES_CLG
```

Above command specifies the target library to be used by the DC for synthesizing the RTL and mapping the logic to appropriate standard cells. For example,

```
assign Z = A && B;
```

DC synthesizes to AND gate which is inferred from the target library.

```
"saed14hvt_ss0p95v125c.db"
```

Exclude the below set of Target libraries during optimization using below "set_dont_use" command.

This command sets the dont_use attribute on library cells, modules, and implementations. Library objects with the dont_use attribute is excluded from the target library during optimization.

```
dc_shell >set_dont_use [get_lib_cells -quiet */SAEDRVT14_A021_U_0P5]  
dc_shell >set_dont_use [get_lib_cells */SAED*_E02_V1_0P75]  
dc_shell >set_dont_use [get_lib_cells */SAED*_A0221_2]  
dc_shell >set_dont_use [get_lib_cells */SAED*_A0221_0P5]  
dc_shell >set_dont_use [get_lib_cells */SAED*_0AI22_0P5]
```

Analyze the specified HDL source files by using below command.

```
dc_shell > analyze -library WORK -format verilog
{../RTL/top_odyssey.v \
 ../RTL/srff.v \
 ../RTL/power_controller.v \
 ../RTL/mult3232.v \
 ../RTL/MemXHier.v \
 ../RTL/MemYHier.v \
 ../RTL/InstructionDecoder.v \
 ../RTL/gpr.v \
 ../RTL/genpp32.v \
 ../RTL/csa.v \
 ../RTL/cla.v \
 ../RTL/address_gen.v \
 ../RTL/addpp32.v \
 }
```

Check for any Errors / Warnings.

This command reads the RTL files and analyses the RTL code and stores the resulting templates into the specified library in a format ready to elaborate to form link-able cells of a full design.

analyze - It translates specified HDL files and stores the intermediate format in the specified library.

Read the Top Module design by using the command "read_file".

```
dc_shell > read_file -format verilog {../RTL/top_odyssey.v}
```

Check for design problems in current design, by using the "check_design" command.

```
dc_shell > check_design
```

Read Synopsys Design Constraints (SDC) file from below path.

```
dc_shell > source -echo ../inputs/chiptop+_s0.sdc
```

Specifies a list of registers in the current design to be included for clock gating.

```
dc_shell > set_clock_gating_registers
-include_instances [remove_from_collection
 [all_registers -clock clock]
 [get_cells "MemYHier/MemXb MemYHier/MemXa MemXHier/MemXb MemXHier/MemXa"]]
```

Define the operating conditions for the current design using below command.

```
dc_shell > set_operating_conditions -min ff0p88v125c
-max ss0p6v125c
```

Now link the design with libraries by using link command.

The link command uses the link_library and search_path variables along with the local_link_library design attribute to resolve design references.

```
dc_shell > link
dc_shell > set_fix_multiple_port_nets -all
-buffer_constants [get_designs ChipTop]
```

Once the design environment & constraint have been defined, it can now be synthesized.
Now after constraining design, do check_timing.

```
dc_shell > check_timing
```

Perform a high effort compile on the current design for the better quality of results (QoR), by using the "compile" command.

```
dc_shell > compile -exact_map -gate_clock
```

Modify all names in the design hierarchy. By default, the tool changes only objects in the current design.

```
dc_shell > change_names -rules verilog -verbose -hier
```

Report of clock gating

```
dc_shell > report_clock_gating
```

Set the fix_hold attribute on clocks in the current design.

```
dc_shell > set_fix_hold [all_clocks]
```

The above command sets the fix_hold attribute on the specified clock objects in the current design. The fix_hold attribute informs optimization that hold time violations of the specified clocks should be fixed.

```
dc_shell > report_constraints -min_delay
```

Perform incremental Compile on the current design using below command.

```
dc_shell > compile -incremental -only_design
```

After synthesis, check if the design meets timing, area, power constraints using following report commands and redirect it to the report's directory.

```
dc_shell > report_timing > ../reports/timing.rpt
dc_shell > report_area > ../reports/area.rpt
dc_shell > report_power > ../reports/power.rpt
```

Now write out the design netlist to a file, by using the below command.

```
dc_shell > write -f verilog -h -out ../outputs/ChipTop.v
```

Check the Netlist file by using below command syntax.

```
dc_shell > sh gvim ../outputs/ChipTop.v &
```

Write out a Synopsys internal database format (DDC) file by using below command for physical design.

```
dc_shell > write -f ddc -h -out ../outputs/ChipTop.ddc
```

Write out a Synopsys Design Constraints (SDC) file by using below command for physical design.

```
dc_shell > write_sdc ../outputs/output.ChipTop.sdc
```

Check the SDC file by using below command.

```
dc_shell > sh gvim ../outputs/output.ChipTop.sdc &
```

Close the DC shell.

```
dc_shell > exit
```